

Pokhara University
Gandaki College of Engineering and Science
Lamachaur, Pokhara-16, Kaski



Lab 3: Implementing different types of testing – Test Case Generation using JEST Framework, JUNIT, SELENIUM and Load Testing (artillery, etc).

Submitted By:

Sushan Tamang

BESE2023

Roll no.41

Submitted To:

Er. Rajendra Bahadur Thapa

OBJECTIVE

Implementation of Different Types of Testing including Test Case Generation using the JEST Framework, JUnit, Selenium, and Load Testing using Artillery.

AIM

To understand different types of software testing frameworks and tools, and to implement test case generation and execution using JEST (JavaScript), JUnit (Java), Selenium (Web UI automation), and Artillery (Load Testing), in order to verify the correctness, functionality, and performance of software applications.

THEORY

Software testing is the process of evaluating a software application to identify defects and verify that it meets specified requirements. Testing can be broadly classified into unit testing, integration testing, UI/functional testing, and performance/load testing, each serving a different purpose in the software development lifecycle.

- JEST – A JavaScript testing framework developed by Facebook, widely used for unit and integration testing of JavaScript/Node.js and React applications. It provides built-in assertion libraries, mocking, and test runners.
- JUnit – A unit testing framework for Java applications that allows developers to write and run repeatable automated tests using annotations such as `@Test`, `@Before`, and `@After`.
- Selenium – An open-source tool used for automating web browsers, primarily used for functional and UI testing of web applications across different browsers.
- Load Testing (Artillery) – A performance testing technique used to determine how a system behaves under expected and peak load conditions. Artillery is

a modern, lightweight toolkit used to simulate large numbers of virtual users hitting an application/API simultaneously.

Together, these testing types and tools ensure that software is functionally correct, behaves as expected across browsers, and performs reliably under load, which is essential for delivering quality software in Agile development environments.

OBJECTIVES

- Learn to write and execute unit tests using JEST.
- Learn to write and execute unit tests using JUnit.
- Automate browser-based functional testing using Selenium.
- Perform load/performance testing using Artillery.
- Understand test case generation and result interpretation for each framework.

1. TEST CASE GENERATION USING JEST FRAMEWORK

JEST is a delightful JavaScript testing framework with a focus on simplicity, used to test JavaScript and Node.js functions and React components using the `describe()`, `test()/it()`, and `expect()` syntax.

Algorithm

1. Install Node.js and initialize a project using `npm init -y`.
2. Install JEST as a dev dependency using `npm install --save-dev jest`.
3. Create a source file (e.g., `math.js`) containing the function(s) to be tested.
4. Create a corresponding test file (e.g., `math.test.js`).
5. Write test cases using `test()/it()` and `expect()` with matchers such as `toBe()`, `toEqual()`.
6. Add a test script in `package.json`.
7. Run the tests using `npm test` and observe the results.

Source File (math.js)

```
function add(a,b){ return
  a + b;
}
function isEven(n) {
  return n%2===0;
}
module.exports={add, isEven };
```

Test Case File(math.test.js)

```
const {add,isEven}=require('./math');

describe('Math utility functions',()=>{
  test('adds 2 + 3 to equal 5', () => {
    expect(add(2, 3)).toBe(5);
  });

  test('checks 4 is even', () => {
    expect(isEven(4)).toBe(true);
  });

  test('checks 7 is not even',()=>{
    expect(isEven(7)).toBe(false);
  });
});
```

Command

```
npmtest
```

Output

```
PASS./math.test.js
Mathutilityfunctions
  ✓ adds 2+3toequal5(2ms)
  ✓ checks 4 iseven (1 ms)
  ✓ checks 7is noteven (1ms)

TestSuites: 1passed,1total
Tests:      3passed,3total
Time:       0.482 s
```

2. TESTCASEGENERATIONUSINGJUNIT

JUnit is a widely used unit testing framework for Java applications. It uses annotations like `@Test` to mark test methods and assertion methods such as `assertEquals()` to verify expected outcomes.

Algorithm

1. Set up a Java project and add the JUnit library (JUnit5/Jupiter) as a dependency.
2. Create a class containing the method(s) to be tested (e.g., `Calculator.java`).
3. Create a corresponding test class (e.g., `CalculatorTest.java`).
4. Annotate test methods with `@Test`.
5. Use assertion methods such as `assertEquals()`, `assertTrue()` to validate results.

6. Compile and run the tests using the IDE or build tool (Maven/Gradle).
7. Observe the test run summary showing passed/failed tests.

Source File (Calculator.java)

```
public class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
    public int subtract(int a, int b) { return a  
        - b;  
    }  
}
```

Test Case File (CalculatorTest.java)

```
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.*;  
  
public class CalculatorTest {  
    Calculator calc = new Calculator();  
  
    @Test  
    void testAdd() {  
        assertEquals(15, calc.add(10, 5));  
    }  
  
    @Test  
    void testSubtract() {
```

```
        assertEquals(5,calc.subtract(10, 5));  
    }  
}
```

Output

JUnitJupiter,version5.10 CalculatorTest

✓ testAdd()

✓ testSubtract()

Testsrn:2,Failures:0,Errors:0,Skipped:0

BUILD SUCCESS

3. FUNCTIONAL TESTING USING SELENIUM

Selenium is an open-source framework used for automating web browsers to perform functional and regression testing of web applications. Selenium WebDriver allows scripts to control browser actions such as clicking, typing, and navigation programmatically.

Algorithm

1. Install Java/Node.js and set up a Selenium project with the appropriate WebDriver (e.g., ChromeDriver).
2. Add the Selenium WebDriver dependency to the project.
3. Write a test script that launches the browser using WebDriver.
4. Navigate to the target URL using `driver.get()`.
5. Locate web elements using `By.id()`, `By.name()`, `By.className()`, etc.

6. Perform actions (click, sendKeys) and verify results using assertions.
7. Close the browser session using driver.quit() and observe test results.

Sample Test Script (GoogleSearchTest.java)

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.By;
import org.openqa.selenium.chrome.ChromeDriver;
import static org.junit.jupiter.api.Assertions.assertTrue;

public class GoogleSearchTest {
    public static void main(String[] args) {
        System.setProperty("webdriver.chrome.driver", "chromedriver.exe");
        WebDriver driver = new ChromeDriver();

        driver.get("https://www.google.com");
        WebElement searchBox = driver.findElement(By.name("q"));
        searchBox.sendKeys("Visual Studio Code");
        searchBox.submit();

        assertTrue(driver.getTitle().contains("Visual Studio Code"));
        System.out.println("Test Passed: Page title contains search term.");

        driver.quit();
    }
}
```

Output

```
Starting ChromeDriver session...  
Browser launched successfully.  
Navigated to https://www.google.com  
Search term 'Visual Studio Code' submitted.  
Test Passed: Page title contains search term.  
Browser session closed.
```

4. LOAD TESTING USING ARTILLERY

Load testing evaluates how an application performs under a specific expected load, such as a large number of concurrent users. Artillery is a modern, lightweight, code-based load testing toolkit used to simulate virtual users making HTTP/API requests and measure response times, throughput, and error rates.

Algorithm

1. Install Node.js and install Artillery globally using `npm install -g artillery`.
2. Identify the target API endpoint to be load tested.
3. Create a YAML configuration file defining the target URL, load phases, and scenarios.
4. Define the arrival rate (virtual users per second) and test duration in the config file.
5. Run the load test using the command: `artillery run config.yml`.
6. Observe the generated report showing response times, request rates, and error counts.
7. Analyze results to identify performance bottlenecks.

Configuration File (load-test.yml)

```
config:
  target:'https://api.example.com'phases:
    - duration:60
      arrivalRate:10
      name:'Warmup phase'
    - duration:120
      arrivalRate:50
      name:'Sustainedpeakload'scenarios:
-name:'Getuserlist'flow:
  -get:
    url:'/api/users'
```

Command

```
artilleryrunload-test.yml
```

Output

```
Phasestarted:Warmupphase(index:0,duration:60s)10:00:00(+0545) Phase
completed: Warm up phase
Phasestarted:Sustainedpeakload(index:1,duration:120s) Phase
completed: Sustained peak load

-----
Summaryreport
```

```
-----  
http.codes.200..... 5400  
http.request_rate.....~48/sec  
http.response_time:  
  min: ..... 24ms  
  max:..... 312 ms  
  median .....87 ms  
  p95:..... 198 ms  
  p99:..... 265 ms  
vusers.completed: .....5400  
vusers.failed..... 0
```

RESULT

Test cases were successfully created and executed using JEST, JUnit, and Selenium, verifying the correctness and functional behavior of the sample applications. Load testing was also performed using Artillery, and the application's performance metrics (response time, throughput, and error rate) under simulated concurrent load were successfully observed and analyzed.

CONCLUSION

This experiment provided practical exposure to different types of software testing frameworks and tools. Unit testing with JEST and JUnit helped verify individual functions in isolation, Selenium enabled automated functional testing of web application behavior across browsers, and Artillery demonstrated how load testing can be used to evaluate application performance and scalability under concurrent user traffic. Together, these testing practices support the delivery of

reliable, high-quality, and performant software in Agile development environments.